



Multi-Tenant AI-Organizations Platform

Delivered by: ASRP · **Role:** Core engineering / #1 contributor (backend, admin dashboard, deployment layer) ·
Engagement type: Contract · **Domain:** Multi-tenant SaaS for business AI agents

1. Executive summary

A multi-tenant SaaS platform for building and running “AI organizations” — an admin dashboard where a business operator defines an AI agent (its purpose, instructions, tools, and knowledge base), then deploys that same agent across several customer-facing channels: an embeddable **website chat widget**, an **Outlook add-in**, and an **autonomous email processor**. The operator never writes code; the platform turns a plain-language description of a job into a configured, deployable agent.

Architecturally it is two cooperating systems inside one umbrella monorepo: a **TypeScript SaaS** (NestJS backend + React admin + the two client surfaces) that owns tenants, users, agents, deployments, and data, and a **Python agent-authoring/execution service** that generates agents from natural language and runs them. The SaaS is the fast, transactional system of record; the Python service is the slow, bursty generation/execution tier. Agent reasoning runs on **Google’s Gemini (via Vertex AI)** and **Anthropic’s Claude (via LiteLLM)**; the surrounding infrastructure is cloud-native (managed search for retrieval/RAG, a service bus for async work, blob storage for documents, a managed email service, and container/static-web hosting in an EU region). It is deployed to production and in active development.

2. Positioning & problem

The product thesis: **small and mid-sized businesses want AI agents doing real back-office work — triaging email, reading documents, answering website visitors — but they can’t staff an ML team and shouldn’t have to.** The answer is an operator-facing control plane. A non-technical admin describes the job in prose, the platform assembles a working agent, and one click deploys it to wherever the work arrives.

Before this platform, standing up such an agent meant bespoke prompt engineering, a custom integration per channel, and a hand-rolled tenant/permissions layer for every customer. The objective was a **single product** where agent definition, knowledge ingestion, tool wiring, multi-tenant isolation, and multi-channel deployment are all first-class, reusable primitives — so onboarding a new business is configuration, not a new project.

This case study is the **operator/SaaS story**; the agent-developer infrastructure it sits on is a separate body of work covered under NDA.

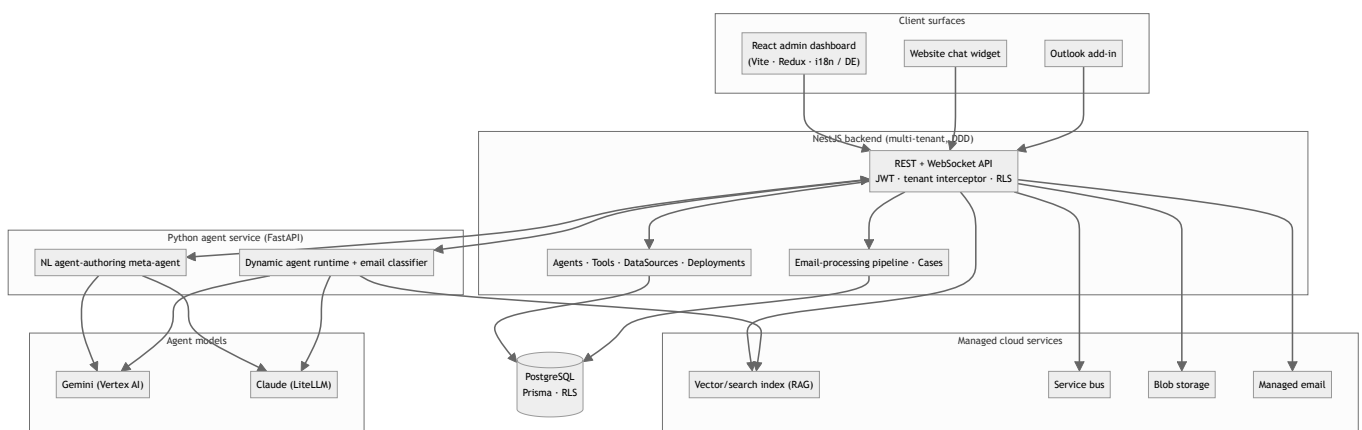
3. What was built

The delivered surface, from the isolation layer up to day-two operations:

- **Multi-tenant control plane** — tenants, invite-only users, roles/status, and per-tenant data isolation enforced at the database layer, with tenant context carried on every request from the JWT.
- **Agent builder** — a guided wizard (profile → data sources → configuration) and a detail workspace (prompt, config, email/screen templates, schedules, sub-agents, relations) plus a live **Test** playground that runs the agent against the Python service, all scoped to a tenant.
- **Natural-language agent authoring** — a Python meta-agent that decomposes a plain-text brief and dispatches specialized sub-makers (config, instructions, plus screen and email-template generation) into a full, validated configuration the SaaS can deploy.
- **Knowledge / RAG** — file-based data sources ingested and indexed into a managed vector/search index (each chunk tenant-tagged), attachable to any agent for grounded answers.
- **Three deployment channels from one agent** — an embeddable website chat widget (with a full per-tenant branding designer), an Outlook add-in, and an autonomous email agent that classifies inbound mail against the tenant's own agents and routes each message accordingly.
- **Email processing pipeline** — stateful, retryable inbound-email handling (PENDING → PROCESSING → PROCESSED / FAILED / MAX_RETRIES_EXCEEDED) with per-tenant, dynamic classification and a source-agnostic **Case** model for tracked work items.
- **Tool integrations & secrets** — a typed tool registry with per-agent, per-tenant encrypted secrets, plus mail-connection credentials for Google/Microsoft mailboxes.
- **Operational surface** — activity/audit logs, execution logs and per-step traces, cron-driven scheduling, WebSocket gateways for live updates, log-retention, and a Swagger-documented API.

4. Architecture

The spine: **React admin dashboard** → **NestJS API (tenant-scoped, DDD-layered)** → **PostgreSQL (Prisma)**, with the NestJS backend orchestrating managed cloud services and delegating agent authoring/execution to the **Python agent service**. The same agent definitions fan out to three deployment surfaces.



Component	Responsibility
React admin	Operator UI: build agents, manage tenants/users, configure deployments, view logs/analytics. Vite + Redux Toolkit + Tailwind + react-i18next (ships German; i18n-ready), live updates over Socket.IO.
NestJS backend	Authoritative API. DDD layering (domain / application / infrastructure), JWT/Passport auth, per-request tenant context, Prisma over PostgreSQL with row-level security, Swagger, cron scheduling, WebSocket gateways.
Python agent service	Agent authoring + execution: an NL meta-agent, a dynamic agent runtime, run/chat endpoints, and the email-processing pipeline.
Chat widget	Embeddable per-tenant website chat surface — a loader that mounts an iframe and talks to the backend over Socket.IO.
Outlook add-in	Brings the agent into the mailbox as an Office.js task-pane add-in.
Agent models	Gemini via Google Vertex AI and Claude via LiteLLM, behind a model allow-list (no silent default).
Managed cloud	Vector/search index (retrieval), service bus (async), blob storage (documents), managed email (send), container / static-web hosting, Microsoft identity.

5. Engineering deep-dives

- **Tenant isolation at the database.** Row-level security (FORCE RLS) plus a per-request tenant context from the JWT means isolation is enforced by the database, not just application code — a mistake in a query can't leak across tenants.
- **One definition, three runtimes.** An agent is defined once; the widget, the Outlook add-in, and the email processor each execute that same definition against the Python service. Adding a channel doesn't fork the agent model.
- **Natural language → configured agent.** The authoring meta-agent decomposes a prose brief and fans out to specialized sub-makers (config, instructions, screen and email templates), returning a validated configuration — turning “describe the job” into a deployable agent in about a minute.
- **Stateful, retryable email.** The inbound pipeline is an explicit state machine with bounded retries and a source-agnostic Case model, so mail processing is observable and recoverable rather than fire-and-forget.
- **Model allow-list.** Reasoning runs on Gemini and Claude behind an explicit allow-list — no silent default model, so cost and behaviour are predictable per tenant.

6. Stack

Layer	Choice
Admin frontend	React + Vite, Redux Toolkit, Tailwind, react-i18next, Socket.IO
Backend	NestJS (DDD-layered), JWT/Passport, Prisma, Swagger, WebSocket gateways, cron
Database	PostgreSQL with row-level security (FORCE RLS)
Agent service	Python + FastAPI — NL authoring meta-agent, dynamic runtime, email pipeline
Models	Gemini (Vertex AI) + Claude (LiteLLM), behind a model allow-list
Cloud	Managed vector/search (RAG), service bus, blob storage, managed email, container/static-web hosting (EU region)

7. Data & interfaces

The core entities are **Tenant, User, Agent, DataSource, Deployment, Tool, and Case**, all tenant-scoped in PostgreSQL under row-level security. Agents carry prompt/config/templates/schedules and can relate to sub-agents. The backend exposes a Swagger-documented REST API plus WebSocket gateways for live updates; the Python service exposes run/chat endpoints the SaaS calls for authoring and execution. Inbound email flows through the Case model with an explicit processing state machine.

8. Reliability & operations

- **Isolation enforced by the DB** (FORCE RLS) — defence in depth beyond app checks.
- **Observability** — activity/audit logs, execution logs, per-step traces, and log-retention.
- **Async & scheduled work** — a service bus for bursty generation/execution and cron-driven schedules.
- **Release flow** — a staging → production promotion path (owned by ASRP), with the Python service scaled independently of the transactional SaaS.

9. Metrics

Metric	Value
Deployment channels per agent	3 (web / Outlook / email)
Agent authoring from prose	~1 min (target)
Contribution (both core repos)	#1, ~45% of commits
Tenant isolation	PostgreSQL FORCE RLS + JWT tenant context

10. Status & roadmap

Status: production, active development. The platform is a hosted multi-tenant SaaS (invite-only). Direction of travel: broader channel coverage, richer authoring, and deeper analytics. The agent-developer infrastructure underneath (SDK, runtime, agent templates) is a separate, NDA-covered body of work.

11. What ASRP delivered

ASRP was the #1 contributor (~45% of commits) across both core repositories: the multi-tenant NestJS backend (tenant isolation, agents/tools/data-sources/deployments, the email-processing pipeline), the React admin dashboard, and the agent-deployment layer that fans one definition out to the widget, Outlook add-in, and email processor. ASRP also owned the staging → production release flow.

12. See it

The platform is a hosted multi-tenant SaaS (invite-only), so there is no anonymous public login, and the source repositories are private. The client is not named here; identifying detail is available under NDA. Screenshots are omitted — all captures from source contained live infrastructure hostnames and tenant data.

13. FAQ (for client conversations)

- **Is this a single-tenant or multi-tenant system?** Multi-tenant, with isolation enforced at the database via row-level security and a per-request tenant context from the JWT.
- **Do operators need to code?** No — an agent is described in plain language and the platform assembles a validated, deployable configuration.
- **How is one agent used across channels?** An agent is defined once and executed against the Python service from three surfaces: website widget, Outlook add-in, and email processor.
- **Which models does it use?** Google's Gemini (via Vertex AI) and Anthropic's Claude (via LiteLLM), behind an explicit model allow-list.
- **Why is this anonymized?** The client is under NDA. Capability-level detail is shown here; identifying specifics are available on request under NDA.

Anonymized client-engagement case study prepared by ASRP for portfolio use. The client is under NDA and is not named; no links are shown. Implementation identifiers, endpoints, credentials, and tenant data are intentionally excluded; capability-level detail only.